

VERILANG ENTREGA 4

Isabella Salcedo Delgadillo - 202420963

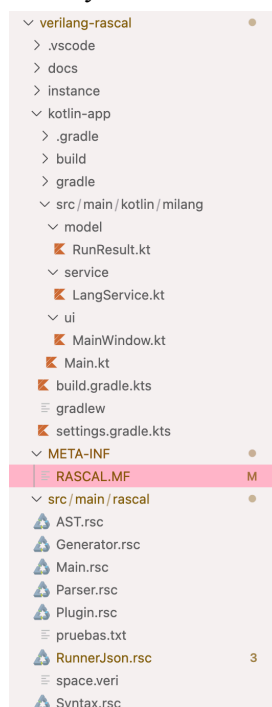
Valentina Rojas Forero - 202420927

NOTA: Los archivos de contenido del programa tienen una extensión `.veri` en vez de `.vl`, pues desde el proyecto anterior manejábamos este tipo de archivos para pruebas, y para evitar posibles errores se decidió dejar así.

A partir del trabajo hecho durante todo el semestre, dentro de esta entrega se conectó la implementación en Rascal del lenguaje VeriLang, incluyendo la gramática, el AST y el verificador de tipos, con una interfaz gráfica desarrollada en Kotlin. El objetivo es que el usuario pueda escoger un archivo `.veri`, ejecutarlo con el parser de Rascal y ver los resultados del análisis en la interfaz.

Para esto, se creó el archivo `RunnerJson.rsc`, siguiendo la plantilla dada, y este está encargado de exportar el resultado del análisis en JSON, y se desarrolló la aplicación Kotlin que consume el JSON y lo muestra en una GUI.

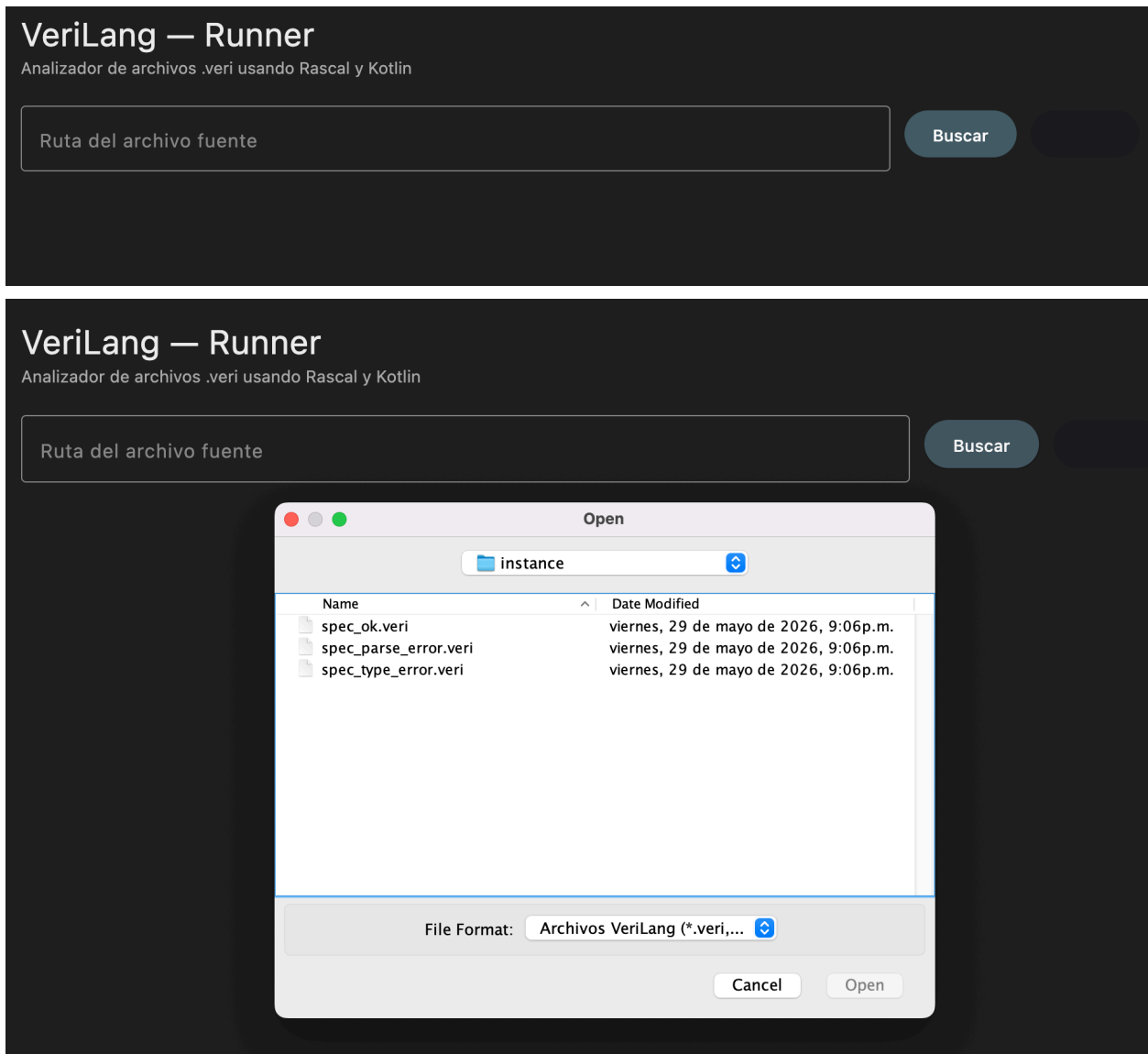
El sistema quedó de la siguiente forma, ya incluyendo Kotlin



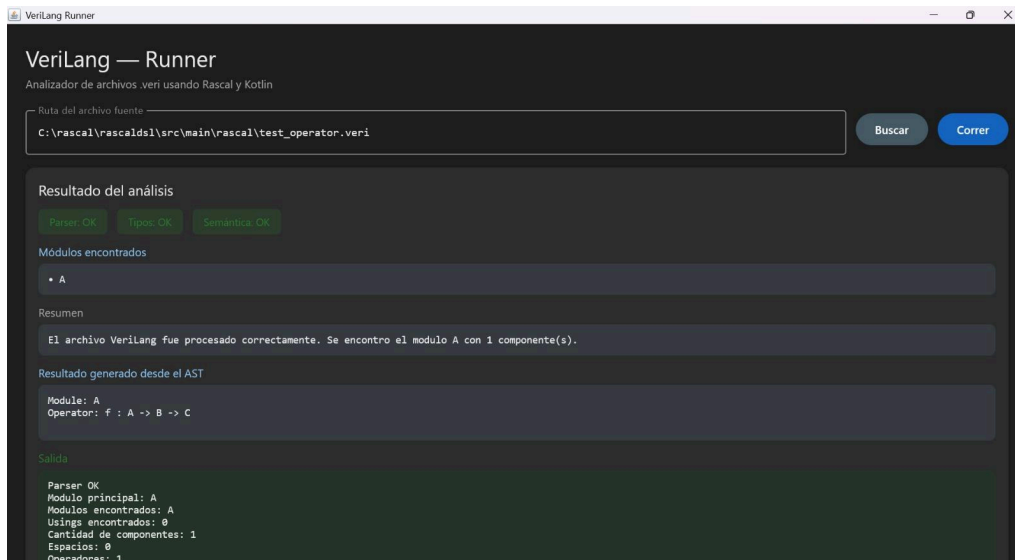
Para poder ejecutarlo, es necesario tener instalado Gradle, y luego correr:

```
cd kotlin-app
gradle run
```

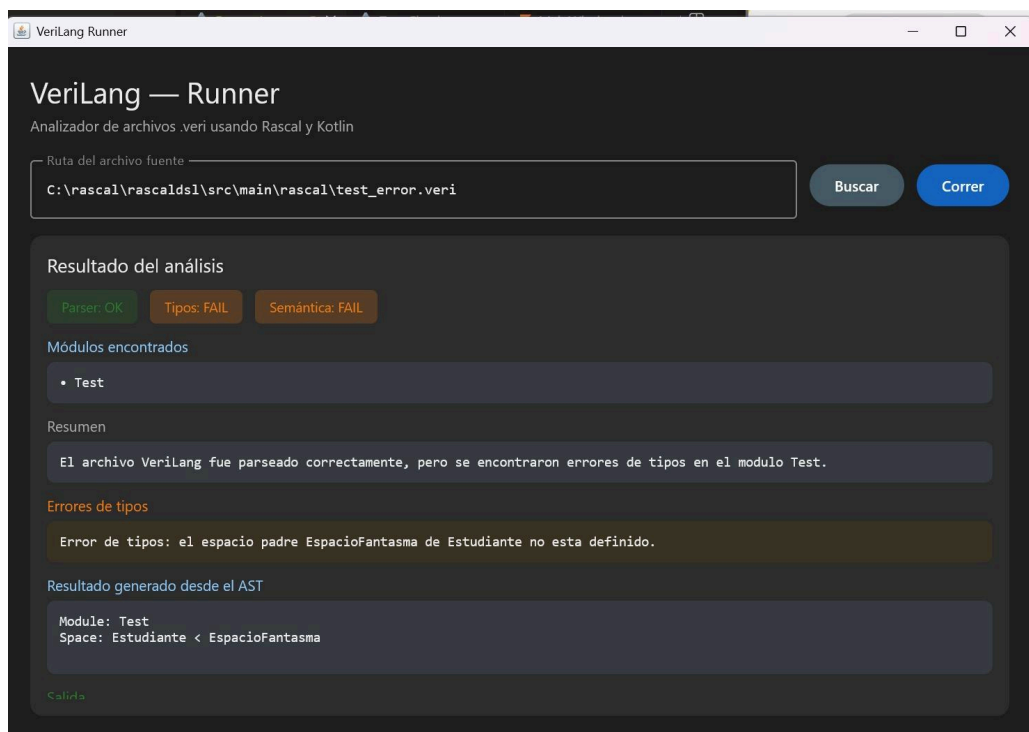
Después, dándole clic a buscar, se puede seleccionar el archivo que se quiere correr. En el proyecto hay varios archivos de prueba dentro del proyecto, con la extensión .veri, para ver el funcionamiento correcto de la aplicación y de la gramática.



Cuando un archivo es válido, el sistema corrobora que tenga espacios definidos, operadores, variables y expresiones válidas, arrojando el siguiente resultado:



En cambio, cuando un archivo presenta errores, como test_error.veri, donde defspace Persona no tiene end en la misma línea, la interfaz lanza el siguiente error:



Cambios respecto al proyecto 3

Con respecto al anterior, en esta entrega se creó RunnerJson.rsc como nuevo módulo de entrada para Kotlin, se agregó collectTypeErrors() en TypeChecker.rsc para retornar errores como lista en lugar de imprimirlos, se renombraron los módulos al paquete milang:: para tener compatibilidad con el runner, se adaptó la aplicación Kotlin completa (RunResult, LangService, MainWindow) para verilang, y se crearon

los archivos de prueba. Todo esto, logró integrar la implementación de VeriLang en Rascal con un entorno de ejecución externo en Kotlin.